■■ Speaker Recognition System

Complete Technical Architecture, File Breakdown & Voice Recognition Flow

Document Version: 1.0	**Target Model:** BiLSTM Embedder
Feature Extractor: MFCC (40) + Δ + Δ²	**Frontend / Backend:** React + Vite / Flask REST API
Sampling Rate: 16,000 Hz Mono	**Open-Set Classifier:** Cosine Voiceprint Centroids

1. Executive Project Overview

This project implements a state-of-the-art **Bi-directional Long Short-Term Memory (BiLSTM) Speaker Recognition and Voice Intelligence Console**. The system is designed to perform end-to-end speaker identification, real-time voice recording analysis, multi-speaker diarization, and unknown speaker rejection (open-set recognition).

Unlike basic closed-set classifiers that force every unknown voice into one of the trained classes with high false confidence, this system extracts a 64-dimensional normalized voiceprint embedding vector on a unit hypersphere. It evaluates cosine similarity against enrolled speaker reference centroids, allowing robust rejection of unenrolled speakers.

2. Detailed File & Module Breakdown

The codebase is divided into clear modular layers: core ML pipeline (src/), web backend API (app/), frontend application (frontend/), and data/model persistence directories.

File / Path	Module Role & Functional Description
`src/config.py`	**Central Hyperparameter Registry:** Sets 16kHz sample rate, 3.0s window length, 1.5s hop step, 40 MFCCs (+ deltas = 120 features), silence trimming threshold (25 dB), speech RMS threshold (0.005), and similarity threshold (0.58).
`src/preprocess_audio.py`	**Audio Preprocessing Pipeline:** Loads WAV/MP3/FLAC via Librosa at 16kHz mono, trims silence, checks RMS energy to reject pure noise, peak-normalizes audio signal, and cuts waveforms into 3-second overlapping segments.
`src/extract_mfcc.py`	**Feature Extraction Engine:** Computes 40 MFCCs per frame (25ms Hann window, 10ms hop), appends 1st and 2nd derivatives (Delta & Delta-Delta), applies Cepstral Mean & Variance Normalization (CMVN), generating a (301, 120) feature matrix per window.
`src/prepare_dataset.py`	**Dataset Builder & Augmenter:** Chunks long speaker files into ~10s clips, creates stratified train/val/test splits, applies 4 audio augmentations (white noise, gain scaling, pitch/speed resampling, combined), generates synthetic background features, and outputs binary .npy files.
`src/train_model.py`	**Deep Learning Trainer:** Constructs a 2-layer BiLSTM model (64 & 32 units) with L2 regularization, Dropout (0.2), and L2 Unit Normalization. Computes 64-dimensional reference voiceprint centroids for all enrolled speakers and saves centroids.npy.
`src/predict.py`	**Inference & Diarization Engine:** Processes single files or live microphone streams, computes segment-level & aggregate cosine similarities against centroids, performs open-set unknown speaker rejection, multi-speaker overlap detection, and timeline generation.

`src/evaluate_model.py`	**Developer Evaluation Suite:** Generates segment-level and recording-level classification reports, precision/recall/F1 matrices, and saves confusion matrix plots to reports/.
`app/api.py`	**Flask REST API Server:** Serves endpoints /api/health, /api/predict (multipart WAV upload handler), and /api/evaluation with thread-safe model locking on port 8000.
`frontend/`	**Dark Cinematic Web Dashboard:** React + Vite single-page application featuring step navigation, audio visualizer, waveform analyzer, probability charts, MFCC heatmap, timeline breakdown, and printable report exporter.

3. End-to-End Voice Recognition Pipeline

The voice identification process converts raw continuous acoustic pressure waves into clean feature matrices, projects them into a deep hyperspherical embedding space, and matches them against voiceprints. The detailed execution flow is structured into 6 sequential phases:

Phase 1: Audio Loading & Signal Preprocessing

- Input: Audio file upload (WAV/MP3/FLAC) or live 5-second microphone recording.
- Resampling: Converted to mono, 16,000 Hz single-channel floating-point array (y).
- Speech Energy Validation: Root Mean Square (RMS) energy is calculated. If RMS < 0.005, audio is flagged as low energy / quiet noise.
- Silence Trimming: Top 25 dB silence is stripped using librosa.effects.trim.
- Peak Normalization: Audio amplitude is scaled so max peak equals 0.95: y = y / peak * 0.95.

Phase 2: Sliding Window Framing & Segmentation

- Fixed Duration: The cleaned waveform is sliced into 3.0-second fixed-length segments (48,000 samples).
- Overlap Hop: Sliced with a 1.0-second hop step during prediction (1.5s during training) to achieve high temporal resolution.
- Short File Padding: If input duration is less than 3.0s (e.g. 1.5s), wrap-padding repeats the signal up to 3.0s instead of padding silence.

Phase 3: Acoustic Feature Extraction (MFCC + Deltas)

- Framing & Windowing: 400-sample window length (25 ms) with 160-sample hop (10 ms) using a Hann window.
- Spectrum & Mel Filterbank: 512-point FFT transformed into 64 Mel-frequency bands (20 Hz to 7600 Hz).
- Log-DCT & MFCCs: Log mel-powers passed through Discrete Cosine Transform (DCT) to yield 40 MFCC coefficients.
- Dynamic Deltas: 1st order derivatives (Delta, velocity) and 2nd order derivatives (Delta-Delta, acceleration) are concatenated (40 + 40 + 40 = 120 features).
- Cepstral Mean & Variance Normalization (CMVN): Standardized per frame: (M - mean) / (std + 1e-8).
- Output Shape: Fixed **(301 frames, 120 features)** per segment.

Phase 4: BiLSTM Neural Network Embedding Generation

- Layer 1: Bidirectional(LSTM(64, return_sequences=True, L2=1e-4)) captures temporal speech dynamics.
- Layer 2: Bidirectional(LSTM(32, return_sequences=False, L2=1e-4)) aggregates whole-segment acoustic context.
- Dense Projection: Dense(64, linear) followed by BatchNormalization().
- Hyperspherical Normalization: UnitNormalization(axis=-1) projects the vector onto a unit sphere (L2 norm = 1).
- Output: A 64-dimensional acoustic **Voiceprint Vector (e)**.

Phase 5: Open-Set Voiceprint Cosine Similarity & Classifier

- Enrolled Reference Centroids: Each speaker's mean vector (c_k) is stored in models/centroids.npy.
- Cosine Similarity Calculation: Dot product between segment embedding e and centroid c_k: Sim(k) = c_k · e.
- Similarity Thresholding: If max similarity < 0.58 (or background prob > 0.50), the voice is rejected as UNKNOWN.
- Dynamic Unknown Clustering: Unenrolled voices are assigned labels ('Unknown Speaker 1') and tracked across time segments.

Phase 6: Multi-Speaker Diarization & Console Rendering

- Overlap Detection: Secondary enrolled speakers with runner-up similarity ≥ 0.46 and margin < 0.18 are tagged as overlapping voices.
- Timeline Assembly: Segment-by-segment predictions create a full chronological timeline.
- Console Output: Flask returns structured JSON data to React frontend for dark cinematic visualization.

4. BiLSTM Model Architecture & Math Formulation

The core neural network relies on a Bidirectional LSTM coupled with a Unit Normalization layer. The mathematical principles governing the feature extraction and classification include:

MFCC Delta Extraction	$\Delta_t = (\Sigma_n n * (c_{t+n} - c_{t-n})) / (2 * \Sigma_n n^2)$ Measures spectral velocity and acceleration across consecutive frames.
L2 Unit Normalization	$e = v / (\|\|v\|\|■ + \varepsilon)$ where $\|\|v\|\|■ = sqrt(\Sigma v_i^2)$ Ensures all voice vectors lie on a unit sphere, making dot product identical to Cosine Similarity.
Speaker Centroid (Voiceprint)	$c_k = Normalize((1 / N_k) * \Sigma_{j \in Speaker_k} e_j)$ Computes the average normalized embedding for speaker *k* across training samples.
Cosine Similarity Metric	$Sim(c_k, e) = c_k \cdot e = \Sigma (c_{k,i} * e_i)$ Returns a value between -1.0 and +1.0 indicating acoustic similarity.

5. Unknown Speaker Rejection & Diarization

Standard softmax classifiers always sum to 1.0, forcing unknown voices to be incorrectly categorized as an enrolled speaker. Our system solves this via dual-verification:

1. **Cosine Similarity Rejection:** Even if softmax probability is high, if the embedding cosine similarity to the closest enrolled centroid is below 0.58, the segment is marked as Unknown.
2. **Background Formant Class:** A synthetic _background_ class is trained on noise/formant patterns. If P(_background_) > 0.50, it is rejected.
3. **Dynamic Unknown Clustering:** When multiple unknown segments occur, their embeddings are compared using cosine similarity. If similarity ≥ 0.65, they are assigned to the same unknown identity ('Unknown Speaker 1').

■ ***System Performance Summary***

- ***Segment-Level Accuracy:*** *~66.85% (3-second window accuracy)*
- ***Recording-Level Accuracy:*** *~76.92% (aggregated full recording accuracy)*
- ***Weighted F1-Score:*** *0.64*
- ***Rejection Precision:*** *> 90% against non-enrolled speakers*
- ***Inference Latency:*** *< 150ms per 5-second audio chunk on CPU*

6. Quick Start & Execution Commands

To train, evaluate, and launch the solution, execute the following commands:

Task / Action	Command Line Instruction
1. Feature Extraction	`python src/prepare_dataset.py --augment`
2. Train BiLSTM Model	`python src/train_model.py --epochs 60 --batch-size 32`
3. Evaluate Model	`python src/evaluate_model.py`
4. CLI Prediction (File)	`python src/predict.py path/to/audio.wav`
5. CLI Prediction (Mic)	`python src/predict.py --mic --seconds 5`
6. Launch Flask API	`python app/api.py`
7. Launch React Web UI	`cd frontend && npm run dev`